Argonne NATIONAL LABORATORY

Dense Broyden–Fletcher–Goldfarb–Shanno (BFGS)

Hansol Suh, Toby Isaac

May 20, 2025

Table of Contents

Overview of BFGS

Dense BFGS

Benchmarks

Overview of BFGS

Quasi-Newton with line search

Algorithm 1: quasi-Newton optimization with line search

1 **Until convergence**
2 $g \leftarrow \nabla f(x)$
3 `B.update`(x, g)
4 `B.solve`(g, p)
5 $x \leftarrow x - \alpha p$, where α is determined by line search on $f(x - \alpha p)$.

What is B?

- `B` approximates Hessian matrix
- `MATLMVM` in PETSc (limited-memory variable metric)
- Used in `SNESQN, PCLMVM, TAOLMVM, ...`

Type	Method	Property
`MATLMVMBFGS`	BFGS	SPD
`MATLMVMDFP`	DFP	SPD
`MATLMVMDBFGS`	Dense BFGS	SPD
`MATLMVMDDFP`	Dense DFP	SPD
`MATLMVMDQN`	Dense QN	SPD
`MATLMVMSR1`	Symmetric Rank-1	Symmetric
`MATLMVMBrdn`	Good Broyden	Square

Table 1: Selected `MATLMVM` types

- BFGS: Broyden-Fletcher-Goldfarb-Shanno
- DFP: Davidon-Fletcher-Powell

Broyden-Fletcher-Goldfarb-Shanno (BFGS)

- Approximates the Hessian (or its inverse) using past secant conditions
- Secant condition: $B_{k+1} \underbrace{(x_{k+1} - x_k)}_{s_k} = \underbrace{\nabla f(x_{k+1}) - \nabla f(x_k)}_{y_k}$
- $$B_{k+1} = B_k + \frac{y_k y_k^T}{y_k^T s_k} - \frac{B_k s_k s_k^T B_k^T}{s_k^T B_k s_k}$$
- Starting from B_0, iteratively do $k+1$ rank one updates

Limited Memory BFGS

- Limited-memory: Stores only a few recent iterates for practical implementation, rather than the entire history - **L-BFGS**, with history size m (e.g., 5, 10, 20)
- Can set initial B_0 - `none, scalar, diagonal, user, decide`
- Will denote classical implementation as `Recursive BFGS`

Algorithm 2: `Recursive::B.solve`(g, p)

1 $m, k, B_0, S, Y, d =$ B.$(\dots)$
2 $w \in \mathbb{R}^{m_k}$ // *temporary work vector*
3 $w \leftarrow g$ // `copy`(n)
4 **for** $i = k-1, k-2, ..., k-m$ **do**
5 $\quad \alpha_i \leftarrow (s_i^T w)/(y_i^T s_i)$ // `dot`(n), `allreduce`(1)
6 $\quad w \leftarrow w - \alpha_i y_i$ // `axpy`(n)
7 $p \leftarrow B_0^{-1} w$ // B_0`.solve()`
8 **for** $i = k-m, k-m+1, ..., k-1$ **do**
9 $\quad \beta \leftarrow (y_i^T p)/(y_i^T s_i)$ // `dot`(n), `allreduce`(1)
10 $\quad p \leftarrow p + (\alpha_i - \beta) s_i$ // `axpy`(n)

L-BFGS

- A lot of sparse BLAS Level 1 operations
- Poor memory movement pattern
- Suboptimal for GPU - frequent synchronization / kernel launches

Dense BFGS

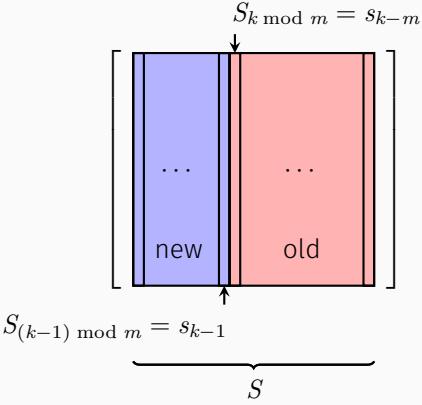
Dense BFGS - Ideas

- Don’t want to iterate over list of vectors with Level-1 BLAS
- Turn data into matrices, and do as much Level-2,3 BLAS as possible

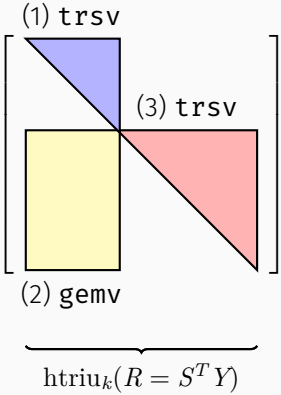
Dense BFGS - Ideas

- Don't want to iterate over list of vectors with Level-1 BLAS
- Turn data into matrices, and do as much Level-2,3 BLAS as possible
- BUT Can't store them in historical order - $O(n)$ vs $O(mn)$ data movement
- Have to store them in *cyclic order* - copy s_k to $S_{k \bmod m}$
- *Cyclic order* doesn't affect `gemv` operations - but affects `trsv`

Cyclic Order



Cyclic doesn't affect **gemv**



Does affect **trsv**

Cyclic Order

- Dense BFGS algorithm requires storage of $R \leftarrow S^T Y$
- We need solve upper triangular part of R - `blas::trsv`
- Because S, Y are in cyclic order, R is also in cyclic order
- Instead of one `trsv`, we get two `trsv`, and one `gemv`

Dense BFGS Solve

Algorithm 3: `Dense::B.solve`(g, p)

1 $m, k, B_0, S, Y, d, R, g^{\text{prev}}, a^{\text{cache}} = \mathsf{B}.(\dots)$
2 $a \in \mathbb{R}^{m_k}$ // *temporary work vector*
3 $\omega \leftarrow \omega(m, k)$ // *active history indices*
4 **if** $g \neq g^{\text{prev}}$ **then**
5 $\quad a \leftarrow S_\omega^T g$ // `gemv`(n, m_k), `reduce`(m_k)
6 **else** $a \leftarrow a_\omega^{\text{cache}}$ // `copy`(m_k)
7 $a \leftarrow \text{htriu}_k(R_{\omega,\omega})^{-1} a$ // `trsv_cyclic`(m_k)
8 $p \leftarrow g - Y_\omega a$ // `gemv`(n, m_k), `bcast`(m_k)
9 $p \leftarrow B_0^{-1} p$ // B_0`.solve()`
10 $a \leftarrow \text{diag}(d_\omega) a$ // `diagm`(m_k)
11 $a \leftarrow a - Y_\omega^T z$ // `gemv`(n, m_k), `reduce`(m_k)
12 $a \leftarrow \text{htriu}_k(R_{\omega,\omega})^{-T} a$ // `trsv_cyclic`(m_k)
13 $p \leftarrow p + S_\omega a$ // `gemv`(n, m_k), `bcast`(m_k)

Dense BFGS Solve

$$H_k^m = \begin{bmatrix} I & SR^{-T} \end{bmatrix} \begin{bmatrix} I & 0 \\ -Y^T & I \end{bmatrix} \begin{bmatrix} H_0 & 0 \\ 0 & D \end{bmatrix} \begin{bmatrix} I & -Y \\ 0 & I \end{bmatrix} \begin{bmatrix} I \\ R^{-1}S^T \end{bmatrix} \tag{1}$$

Dense BFGS Solve

$$H_k^m = \begin{bmatrix} I & SR^{-T} \end{bmatrix} \begin{bmatrix} I & 0 \\ -Y^T & I \end{bmatrix} \begin{bmatrix} H_0 & 0 \\ 0 & D \end{bmatrix} \begin{bmatrix} I & -Y \\ 0 & I \end{bmatrix} \begin{bmatrix} I \\ R^{-1}S^T \end{bmatrix}$$

---

1 $w \leftarrow g$
2 for $i = k-1, k-2, \ldots, k-m$ do
3 $\quad \alpha_i \leftarrow (s_i^T w)/(y_i^T s_i)$;
4 $\quad w \leftarrow w - \alpha_i y_i$
5 $p \leftarrow B_0^{-1} w$
6 for $i = k-m, k-m+1, \ldots, k-1$ do
7 $\quad \beta \leftarrow (y_i^T p)/(y_i^T s_i)$
8 $\quad p \leftarrow p + (\alpha_i - \beta) s_i$

- $R := \text{htriu}(S^T Y)$
- $D := \text{diag}(S^T Y)$

Dense BFGS Solve

$$H_k^m = \begin{bmatrix} I & SR^{-T} \end{bmatrix} \begin{bmatrix} I & 0 \\ -Y^T & I \end{bmatrix} \begin{bmatrix} H_0 & 0 \\ 0 & D \end{bmatrix} \begin{bmatrix} I & -Y \\ 0 & I \end{bmatrix} \begin{bmatrix} I \\ R^{-1}S^T \end{bmatrix}$$

---

1 $w \leftarrow g$
2 for $\underline{i = k-1, k-2, \ldots, k-m}$ do
3 $\quad \alpha_i \leftarrow (s_i^T w)/(y_i^T s_i)$;
4 $\quad w \leftarrow w - \alpha_i y_i$
5 $p \leftarrow B_0^{-1} w$
6 for $\underline{i = k-m, k-m+1, \ldots, k-1}$ do
7 $\quad \beta \leftarrow (y_i^T p)/(y_i^T s_i)$
8 $\quad p \leftarrow p + (\alpha_i - \beta) s_i$

- $R := \text{htriu}(S^T Y)$
- $D := \text{diag}(S^T Y)$

Dense BFGS Solve

$$H_k^m = \begin{bmatrix} I & SR^{-T} \end{bmatrix} \begin{bmatrix} I & 0 \\ -Y^T & I \end{bmatrix} \begin{bmatrix} H_0 & 0 \\ 0 & D \end{bmatrix} \begin{bmatrix} I & -Y \\ 0 & I \end{bmatrix} \begin{bmatrix} I \\ R^{-1}S^T \end{bmatrix}$$

$w \leftarrow g$
for $\underline{i = k-1, k-2, \ldots, k-m}$ do
  $\alpha_i \leftarrow (s_i^T w)/(y_i^T s_i)$;
  $w \leftarrow w - \alpha_i y_i$
$p \leftarrow B_0^{-1} w$
for $\underline{i = k-m, k-m+1, \ldots, k-1}$ do
  $\beta \leftarrow (y_i^T p)/(y_i^T s_i)$
  $p \leftarrow p + (\alpha_i - \beta) s_i$

- $R := \text{htriu}(S^T Y)$
- $D := \text{diag}(S^T Y)$

Dense BFGS Solve

$$H_k^m = \begin{bmatrix} I & SR^{-T} \end{bmatrix} \begin{bmatrix} I & 0 \\ -Y^T & I \end{bmatrix} \begin{bmatrix} H_0 & 0 \\ 0 & D \end{bmatrix} \begin{bmatrix} I & -Y \\ 0 & I \end{bmatrix} \begin{bmatrix} I \\ R^{-1}S^T \end{bmatrix}$$

---

1 $w \leftarrow g$
2 for $\underline{i = k-1, k-2, \ldots, k-m}$ do
3 $\quad \alpha_i \leftarrow (s_i^T w)/(y_i^T s_i)$;
4 $\quad w \leftarrow w - \alpha_i y_i$
5 $p \leftarrow B_0^{-1} w$
6 for $\underline{i = k-m, k-m+1, \ldots, k-1}$ do
7 $\quad \beta \leftarrow (y_i^T p)/(y_i^T s_i)$
8 $\quad p \leftarrow p + (\alpha_i - \beta) s_i$

- $R := \text{htriu}(S^T Y)$
- $D := \text{diag}(S^T Y)$

Types of Dense BFGS Algorithm

- The original dense BFGS was introduced in Byrd et al, 1994[1]

[1] Byrd, Nocedal, and Schnabel 1994

Types of Dense BFGS Algorithm

- The original dense BFGS was introduced in Byrd et al, 1994[1]
- $$H_k^m = H_0 + \begin{bmatrix} S & H_0 Y \end{bmatrix} \begin{bmatrix} R^{-T}(Y^T H_0 Y + D)R^{-1} & -R^{-T} \\ -R^{-1} & 0 \end{bmatrix} \begin{bmatrix} S^T \\ Y^T H_0^T \end{bmatrix}.$$

[1] Byrd, Nocedal, and Schnabel 1994

Types of Dense BFGS Algorithm

- The original dense BFGS was introduced in Byrd et al, 1994[1]
- $$H_k^m = H_0 + \begin{bmatrix} S & H_0 Y \end{bmatrix} \begin{bmatrix} R^{-T}(Y^T H_0 Y + D)R^{-1} & -R^{-T} \\ -R^{-1} & 0 \end{bmatrix} \begin{bmatrix} S^T \\ Y^T H_0^T \end{bmatrix}.$$
- If H_0 is variable, the algorithm requires extra m evaluations of H_0
- `constant`, `scalar` H_0 requires two additional Level-1 BLAS Calls
- Our method avoids this

[1] Byrd, Nocedal, and Schnabel 1994

Recursive vs. Dense BFGS

Algorithm 4: `Recursive`

$w \leftarrow g$
for $\underline{i = k-1, k-2, \ldots, k-m}$ do
  $\alpha_i \leftarrow (s_i^T w)/(y_i^T s_i)$ // `dot`(n), `allreduce`(1)
  $w \leftarrow w - \alpha_i y_i$ // `axpy`(n)
$p \leftarrow B_0^{-1} w$
for $\underline{i = k-m, k-m+1, \ldots, k-1}$ do
  $\beta \leftarrow (y_i^T p)/(y_i^T s_i)$ // `dot`(n), `allreduce`(1)
  $p \leftarrow p + (\alpha_i - \beta) s_i$ // `axpy`(n)

Algorithm 5: `Dense`

$\iota \leftarrow \iota(m, k)$ // *active history indices*
$a \leftarrow S_\iota^T g$ // `gemv`(n, m_k), `reduce`(m_k)
$a \leftarrow \mathrm{htriu}_k(R_{\iota,\iota})^{-1} a$ // `trsv_cyclic`(m_k)
$p \leftarrow g - Y_\iota a$ // `gemv`(n, m_k), `bcast`(m_k)
$p \leftarrow B_0^{-1} p$ // B_0`.solve()`
$a \leftarrow \mathrm{diag}(d_\iota) a$ // `diagm`(m_k)
$a \leftarrow a - Y_\iota^T p$ // `gemv`(n, m_k), `reduce`(m_k)
$a \leftarrow \mathrm{htriu}_k(R_{\iota,\iota})^{-T} a$ // `trsv_cyclic`(m_k)
$p \leftarrow p + S_\iota a$ // `gemv`(n, m_k), `bcast`(m_k)

Recursive vs. Dense BFGS

Algorithm 6: Recursive

$w \leftarrow g$
for $i = k-1, k-2, \ldots, k-m$ do
  $\alpha_i \leftarrow (s_i^T w)/(y_i^T s_i)$ // dot(n), allreduce(1)
  $w \leftarrow w - \alpha_i y_i$ // axpy(n)
$p \leftarrow B_0^{-1} w$
for $i = k-m, k-m+1, \ldots, k-1$ do
  $\beta \leftarrow (y_i^T p)/(y_i^T s_i)$ // dot(n), allreduce(1)
  $p \leftarrow p + (\alpha_i - \beta) s_i$ // axpy(n)

Algorithm 7: Dense

$\iota \leftarrow \iota(m, k)$ // *active history indices*
$a \leftarrow S_\iota^T g$ // gemv(n, m_k), reduce(m_k)
$a \leftarrow \mathrm{htriu}_k(R_{\iota,\iota})^{-1} a$ // trsv_cyclic(m_k)
$p \leftarrow g - Y_\iota a$ // gemv(n, m_k), bcast(m_k)
$p \leftarrow B_0^{-1} p$ // B_0.solve()
$a \leftarrow \mathrm{diag}(d_\iota) a$ // diagm(m_k)
$a \leftarrow a - Y_\iota^T p$ // gemv(n, m_k), reduce(m_k)
$a \leftarrow \mathrm{htriu}_k(R_{\iota,\iota})^{-T} a$ // trsv_cyclic(m_k)
$p \leftarrow p + S_\iota a$ // gemv(n, m_k), bcast(m_k)

Recursive vs. Dense BFGS

Algorithm 8: `Recursive`

$w \leftarrow g$
for $\underline{i = k-1, k-2, \ldots, k-m}$ do
  $\alpha_i \leftarrow (s_i^T w)/(y_i^T s_i)$ // `dot`(n), `allreduce`(1)
  $w \leftarrow w - \alpha_i y_i$ // `axpy`(n)
$p \leftarrow B_0^{-1} w$
for $\underline{i = k-m, k-m+1, \ldots, k-1}$ do
  $\beta \leftarrow (y_i^T p)/(y_i^T s_i)$ // `dot`(n), `allreduce`(1)
  $p \leftarrow p + (\alpha_i - \beta) s_i$ // `axpy`(n)

Algorithm 9: `Dense`

$\iota \leftarrow \iota(m, k)$ // *active history indices*
$a \leftarrow S_\iota^T g$ // `gemv`(n, m_k), `reduce`(m_k)
$a \leftarrow \mathrm{htriu}_k(R_{\iota,\iota})^{-1} a$ // `trsv_cyclic`(m_k)
$p \leftarrow g - Y_\iota a$ // `gemv`(n, m_k), `bcast`(m_k)
$p \leftarrow B_0^{-1} p$ // B_0`.solve()`
$a \leftarrow \mathrm{diag}(d_\iota) a$ // `diagm`(m_k)
$a \leftarrow a - Y_\iota^T p$ // `gemv`(n, m_k), `reduce`(m_k)
$a \leftarrow \mathrm{htriu}_k(R_{\iota,\iota})^{-T} a$ // `trsv_cyclic`(m_k)
$p \leftarrow p + S_\iota a$ // `gemv`(n, m_k), `bcast`(m_k)

Recursive vs. Dense BFGS

Algorithm 10: `Recursive`

$w \leftarrow g$
for $i = k-1, k-2, \ldots, k-m$ do
  $\alpha_i \leftarrow (s_i^T w)/(y_i^T s_i)$ // `dot`(n), `allreduce`(1)
  $w \leftarrow w - \alpha_i y_i$ // `axpy`(n)
$p \leftarrow B_0^{-1} w$
for $i = k-m, k-m+1, \ldots, k-1$ do
  $\beta \leftarrow (y_i^T p)/(y_i^T s_i)$ // `dot`(n), `allreduce`(1)
  $p \leftarrow p + (\alpha_i - \beta) s_i$ // `axpy`(n)

Algorithm 11: `Dense`

$\iota \leftarrow \iota(m, k)$ // *active history indices*
$a \leftarrow S_\iota^T g$ // `gemv`(n, m_k), `reduce`(m_k)
$a \leftarrow \mathrm{htriu}_k(R_{\iota,\iota})^{-1} a$ // `trsv_cyclic`(m_k)
$p \leftarrow g - Y_\iota a$ // `gemv`(n, m_k), `bcast`(m_k)
$p \leftarrow B_0^{-1} p$ // B_0`.solve()`
$a \leftarrow \mathrm{diag}(d_\iota) a$ // `diagm`(m_k)
$a \leftarrow a - Y_\iota^T p$ // `gemv`(n, m_k), `reduce`(m_k)
$a \leftarrow \mathrm{htriu}_k(R_{\iota,\iota})^{-T} a$ // `trsv_cyclic`(m_k)
$p \leftarrow p + S_\iota a$ // `gemv`(n, m_k), `bcast`(m_k)

Recursive vs. Dense BFGS

Algorithm 12: `Recursive`

$w \leftarrow g$
for $i = k-1, k-2, \ldots, k-m$ do
  $\alpha_i \leftarrow (s_i^T w)/(y_i^T s_i)$ // `dot`(n), `allreduce`(1)
  $w \leftarrow w - \alpha_i y_i$ // `axpy`(n)
$p \leftarrow B_0^{-1} w$
for $i = k-m, k-m+1, \ldots, k-1$ do
  $\beta \leftarrow (y_i^T p)/(y_i^T s_i)$ // `dot`(n), `allreduce`(1)
  $p \leftarrow p + (\alpha_i - \beta) s_i$ // `axpy`(n)

Algorithm 13: `Dense`

$\iota \leftarrow \iota(m, k)$ // *active history indices*
$a \leftarrow S_\iota^T g$ // `gemv`(n, m_k), `reduce`(m_k)
$a \leftarrow \mathrm{htriu}_k(R_{\iota,\iota})^{-1} a$ // `trsv_cyclic`(m_k)
$p \leftarrow g - Y_\iota a$ // `gemv`(n, m_k), `bcast`(m_k)
$p \leftarrow B_0^{-1} p$ // B_0`.solve()`
$a \leftarrow \mathrm{diag}(d_\iota) a$ // `diagm`(m_k)
$a \leftarrow a - Y_\iota^T p$ // `gemv`(n, m_k), `reduce`(m_k)
$a \leftarrow \mathrm{htriu}_k(R_{\iota,\iota})^{-T} a$ // `trsv_cyclic`(m_k)
$p \leftarrow p + S_\iota a$ // `gemv`(n, m_k), `bcast`(m_k)

Dense BFGS Comparison

Table 2: Quasi-Newton update performance (`B.update`(x_k, g_k) + `B.solve`(g_k, p))

	B_0^{-1}	BLAS memops			MPI synchronizations		
		Lvl. 1	Lvl. 2	Lvl. 3	`allr.`	`reduce`	`bcast`
`Recursive`	1	$\sim 10mn$	0	0	$2m$	0	0
`Dense`	1	$\sim 2n$	$\sim 4mn$	0	0	3 (2)	2

What about `Mult`?

$$B_k^m = B_0 - \begin{bmatrix} Y & \mathring{P} \end{bmatrix} \begin{bmatrix} -D & L^T \\ L & \mathring{F} \end{bmatrix}^{-1} \begin{bmatrix} Y^T \\ \mathring{P}^T \end{bmatrix}, \tag{2}$$

$\mathring{P}$	$\mathbb{R}^{N\times m}$ matrix to store $B_0 S$
$\mathring{F}$	$\mathbb{R}^{m\times m}$ matrix to store $\mathring{P}^T S$
L	$\mathbb{R}^{m\times m}$ matrix to store $\mathrm{htril}_k(S^T Y)$
J	$\mathbb{R}^{m\times m}$ matrix to store the Cholesky factor $JJ^T = \mathring{F} + L\,\mathrm{diag}(d)^{-1} L^T$

Table 3: Data structure for `B.mult`

What about `Mult`?

- Requires Cholesky factorization
- Cholesky can be unstable, for some B_0 types
- Default is to use classical `Recursive.mult`
- `-mat_lbfgs_recursive <1,0>`

DQN, DFP

- Davidon-Fletcher-Powell (DFP): dual to BFGS
- `BFGS.Solve` := `DFP.Mult`
- `DFP.Solve` := `BFGS.Mult`
- `MATLMVMDQN`: Uses `BFGS.Solve` and `DFP.Mult`
- Computationally, fastest. BUT, has some duality gap. Proceed with caution!

Benchmarks

MatSolve Benchmark

```
1   // src/ksp/ksp/utils/lmvm/tests/solve_performance.c
2
3   MatLMVMGetHistorySize(B, &history_size);
4   MatLMVMUpdate(B, x, g);
5   PetscCall(PetscLogStagePush(matsolve_loop);
6   for (PetscInt iter = 0; iter < n_iters; iter++, xscale *= -1.0, fscale *= -1.0) {
7     VecAXPY(x, xscale, dx);
8     VecAXPY(g, fscale, df);
9     MatLMVMUpdate(B, x, g);
10    MatSolve(B, g, p);
11  }
12  PetscCall(MatLMVMReset(B, PETSC_FALSE));
13  PetscLogStagePop();
```

MatSolve Benchmark - ALCF Polaris CPU

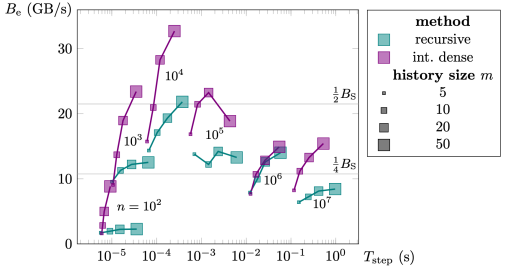


Fig. 2: L-BFGS comparison, Polaris (EPYC 7543P), 1 MPI process

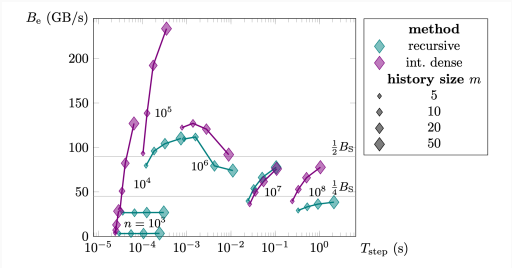


Fig. 4: L-BFGS comparison, Polaris EPYC 7543P, 8 MPI processes (1 per L3 cache)

MatSolve Benchmark - OLCF Frontier CPU

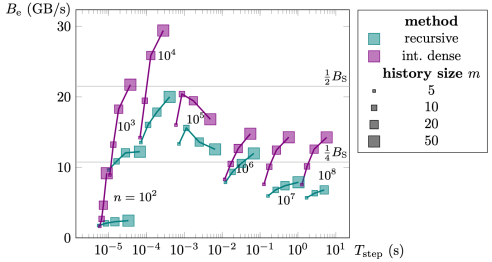


Fig. 3: L-BFGS comparison, Frontier (EPYC 7A53), 1 MPI process

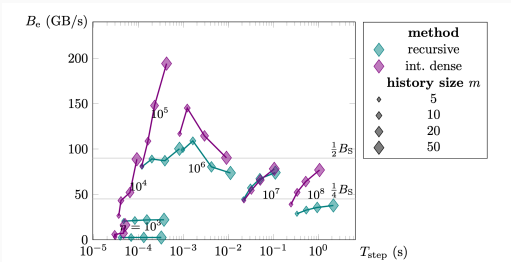


Fig. 5: L-BFGS comparison, Frontier EPYC 7A53, 8 MPI processes (1 per L3 cache)

MatSolve Benchmark - ALCF Polaris GPU

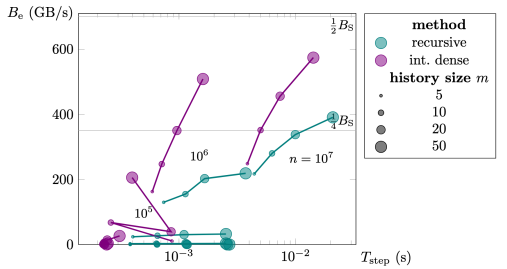


Fig. 6: L-BFGS comparison, Polaris A100, 1 GPU

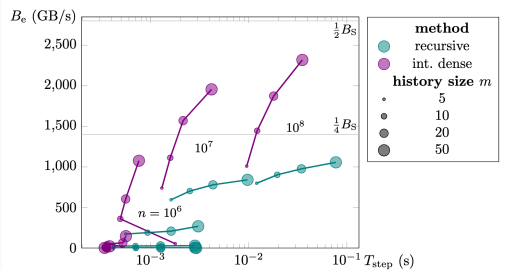


Fig. 8: L-BFGS comparison, Polaris A100, 4 GPUs

MatSolve Benchmark - OLCF Frontier GPU

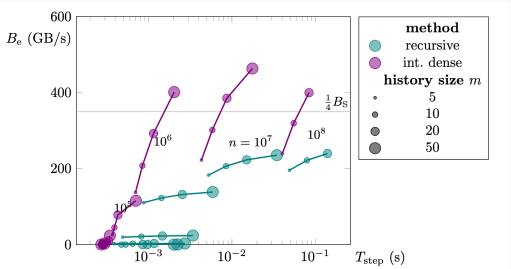


Fig. 7: L-BFGS comparison, Frontier MI250X, 1 GPU

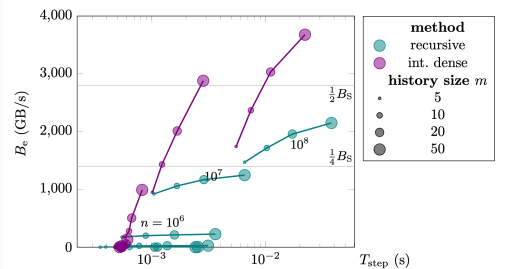


Fig. 9: L-BFGS comparison, Frontier MI250X, 8 GPUs

Summary

- Dense BFGS for quasi-Newton
- Faster and versatile for both CPU/GPU
- `Mult` has Cholesky bottleneck - may use Dense DFP, if needed
- LMVM-DQN: BFGS for `Solve`, DFP for `Mult`
- General improvements on existing recursive methods too